

Rapport de projet PFA

Jeu de type Tower Defense en OCaml

Ufuk Emre YILDIRIM

Avril 2026

Table des matières

1	Présentation du jeu et manuel d'utilisation	2
1.1	Description générale	2
1.2	Les trois types de tours	2
1.3	Flux de jeu	2
1.4	Manuel d'utilisation	3
2	Architecture générale du code et types principaux	4
2.1	Organisation du projet	4
2.2	Types de jeu : <code>cst.ml</code> et <code>global.ml</code>	4
2.3	Archetypes et entités concrètes : <code>component_defs.ml</code>	5
3	Point technique : Résolution des cycles de dépendances	6
3.1	Contexte	6
3.2	Premier cycle : système d'attaque et projectiles	6
3.3	Second cycle : coordinateur de vagues et spawning d'ennemis	6
3.4	Bilan	7
4	Tests effectués	8
4.1	Méthode de développement et de test	8
4.2	Tests manuels par composant	8
4.3	Tests du système de vagues	8
4.4	Test du sous-typage structurel	9
4.5	Problèmes rencontrés non résolus	9

1 Présentation du jeu et manuel d'utilisation

1.1 Description générale

Le projet est un **jeu de type Tower Defense** développé en OCaml, compilé vers JavaScript via `js_of_ocaml` et exécutable dans un navigateur web. L'objectif est de défendre un *noyau central* (le *Core*) contre des vagues successives d'ennemis en plaçant stratégiquement des tours sur la carte. Le noyau possède 500 points de vie ; si un ennemi parvient à le détruire, la partie est perdue.

La carte est une grille de 1200×700 pixels représentant un terrain herbeux. Les ennemis apparaissent aléatoirement sur les quatre bords de la carte et marchent directement vers le noyau. Chaque vague est plus difficile que la précédente : la vague n fait apparaître $5 + 3n$ ennemis.

1.2 Les trois types de tours

Archer (touche 1, coût : 20\$) La tour Archer tire des projectiles verts sur l'ennemi le plus proche dans un rayon de 150 pixels. Chaque projectile inflige 15 points de dégâts et voyage à vitesse fixe. La cadence de tir est d'une flèche toutes les 60 images (≈ 1 seconde à 60 fps).

Bombardier (touche 2, coût : 40\$) La tour Bombardier effectue une explosion de zone (*AoE*) centrée sur une position cible définie par l'utilisateur (clic droit). Elle inflige 30 points de dégâts à tous les ennemis dans un rayon de 50 pixels, toutes les 180 images (≈ 3 secondes). La zone d'explosion est affichée en surbrillance rouge au moment de l'attaque.

Geleur (touche 3, coût : 30\$) La tour Geleur ralentit continuellement tous les ennemis dans un rayon de 120 pixels. Les ennemis gelés voient leur vitesse réduite à 30 % de leur vitesse normale (facteur `freezer_slow_factor` = 0.3). La zone d'effet est visible en permanence sous la forme d'un disque bleu semi-transparent. Les ennemis qui sortent de la zone retrouvent immédiatement leur vitesse normale, et leur cadence d'attaque redevient normale.

1.3 Flux de jeu

Le jeu alterne entre deux phases :

1. **Phase de construction** (*Build mode*) : le joueur dispose d'un budget en argent pour placer des tours. Un panneau en bas de l'écran affiche le budget courant, le numéro de la vague suivante et le raccourci pour démarrer.
2. **Phase de défense** (*Defense mode*) : les ennemis se spawent progressivement (un toutes les 60 images) jusqu'à épuisement de la vague. Le panneau inférieur disparaît. Quand tous les ennemis sont morts, le jeu retourne en phase de construction automatiquement.

1.4 Manuel d'utilisation

Action	Commande
Sélectionner la tour Archer	1
Sélectionner le Bombardier	2
Sélectionner le Geleur	3
Annuler la sélection	0
Placer la tour sélectionnée	Clic gauche
Définir la cible du Bombardier	Clic droit
Lancer la vague suivante	Espace

La prévisualisation de la tour suit le curseur lors du placement. Il n'est pas possible de placer une tour si le budget est insuffisant ou si l'emplacement est déjà occupé par une autre entité.

2 Architecture générale du code et types principaux

2.1 Organisation du projet

```
projet/  
  lib/          -- Bibliothèques ECS et graphique  
  src/  
    core/       -- cst.ml, global.ml, input.ml, wave.ml  
    components/ -- building.ml, soldier.ml, proj.ml  
    systems/    -- attack.ml, draw.ml, move.ml, collision.ml  
  prog/        -- Point d'entrée JS et SDL
```

Tout le code produit se trouve dans `src/`. Les bibliothèques `lib/ecs` (Entity, Component, System) et `lib/gfx` (rendu graphique) étaient fournies et constituent le cadre conceptuel ECS dans lequel le jeu a été développé.

2.2 Types de jeu : `cst.ml` et `global.ml`

Le fichier `cst.ml` centralise toutes les constantes et types fondamentaux.

```
1 (* Les trois types de tours disponibles *)  
2 type buildings = Archer | Bomber of Vector.t | Freezer  
3  
4 (* Etat d'un ennemi *)  
5 type state = Normal | Freeze  
6  
7 (* Quelques constantes de gameplay *)  
8 let archer_radius = 150.0   let archer_cooldown = 60.0  
9 let bomber_radius = 50      let bomber_damage   = 30.0  
10 let freezer_radius = 120    let freezer_slow_factor = 0.3
```

Listing 1 – Types principaux dans `cst.ml`

Le type `Bomber of Vector.t` est notable : il embarque directement la position cible dans le constructeur lui-même, ce qui évite la création d'un composant séparé pour stocker cette information.

L'état global du jeu est centralisé dans le type `Global.t` :

```
1 type mode = Build of Cst.buildings option | Defense  
2  
3 type t = {  
4   window : Gfx.window; ctx : Gfx.context; font : Gfx.font;  
5   money : int; wave : int; mode : mode;  
6   mouse_pos : int * int;  
7   grass : Gfx.surface;  
8   archer_img : Gfx.surface; bomber_img : Gfx.surface;  
9   freezer_img : Gfx.surface; core_img : Gfx.surface;  
10  enemy_img : Gfx.surface;  
11 }
```

Listing 2 – Etat global dans `global.ml`

Le type `mode` encode la phase courante de jeu : en mode `Build`, le champ optionnel indique la tour sélectionnée (`None` si aucune). En mode `Defense` le panneau d'interface disparaît et la vague progresse.

2.3 Archetypes et entités concrètes : component_defs.ml

Ce fichier est le cœur du modèle ECS côté jeu. Il définit les *archetypes* (interfaces attendues par les systèmes) et les classes concrètes qui les implémentent.

Le type tag extensible Pour typer les entités sans couplage entre modules, on utilise un type somme extensible. Chaque module peut ajouter ses propres constructeurs :

```
1 type tag = ..
2 type tag += No_tag | Projectile | Core | Wall
3 type tag += Tower of Cst.buildings
4 type tag += Enemy of Cst.state | Ally
```

Listing 3 – Tags extensibles dans component_defs.ml

Le constructeur `Enemy of Cst.state` est particulièrement utile : il encode à la fois la nature de l'entité et son état courant (gelé ou non), évitant un composant séparé.

Les archetypes Un archetype est un `class type` OCaml regroupant plusieurs composants. Le compilateur vérifie statiquement qu'une entité concrète satisfait l'interface.

```
1 class type collidable = object
2   inherit interactable (* health, timer, tag *)
3   inherit position; inherit box
4   inherit resolver      (* callback appele lors d'une collision *)
5 end
6
7 class type attackable = object
8   inherit Entity.t
9   inherit position; inherit box; inherit health
10  inherit tagged;   inherit timer
11 end
```

Listing 4 – Archetypes collidable et attackable

Classes concrètes Trois classes concrètes implémentent ces interfaces en héritant de tous les composants nécessaires :

```
1 class building () = object
2   inherit Entity.t ()
3   inherit position (); inherit box ();   inherit texture ()
4   inherit tagged ();   inherit resolver (); inherit health ()
5   inherit timer ()
6 end
7
8 class soldier () = object
9   inherit Entity.t ()
10  inherit position (); inherit box ();   inherit texture ()
11  inherit velocity (); inherit tagged (); inherit resolver ()
12  inherit health ();   inherit timer ()
13 end
```

Listing 5 – Classes concretes building et soldier

La classe `projectile` réutilise le champ `health` comme stockage du dommage infligé, satisfaisant ainsi l'interface `collidable` sans composant supplémentaire.

3 Point technique : Résolution des cycles de dépendances

3.1 Contexte

Lors du développement, deux cycles de dépendances entre modules ont bloqué la compilation. En OCaml, les cycles entre modules de la même bibliothèque ne sont pas autorisés (contrairement à des langages comme Java). Les résoudre a nécessité des modifications architecturales précises.

3.2 Premier cycle : système d'attaque et projectiles

Problème Le module `Attack` (logique d'attaque des tours) doit créer des projectiles via `Proj.create`. Or, `Proj` enregistre les projectiles dans les systèmes ECS (`Move_system`, `Draw_system`, `Collision_system`), qui sont eux-mêmes définis dans `System_defs`, qui à son tour importe `Attack`. Cela forme le cycle :

$$\text{System_defs} \rightarrow \text{Attack} \rightarrow \text{Proj} \rightarrow \text{System_defs}$$

Solution La solution consiste à extraire la définition d'`Attack_system` dans un fichier séparé `attack_system.ml`, qui n'est pas inclus dans `System_defs`. `Proj` peut alors dépendre de `System_defs` sans créer de cycle, et `Attack` est importé par `attack_system.ml` indépendamment.

```
1 open Ecs
2 include System.Make(Attack)
3 (* Attack lui-même peut importer Proj et System_defs librement *)
```

Listing 6 – `attack_system.ml` : isolation du cycle

```
1 module Draw_system      = System.Make(Draw)
2 module Move_system      = System.Make(Move)
3 module Collision_system = System.Make(Collision)
4 (* Attack_system est défini à part *)
```

Listing 7 – `system_defs.ml` : sans `Attack_system`

Le graphe de dépendance devient alors acyclique : $\text{System_defs} \rightarrow \{\text{Draw}, \text{Move}, \text{Collision}\}$, $\text{Attack_system} \rightarrow \text{Attack} \rightarrow \text{Proj} \rightarrow \text{System_defs}$.

3.3 Second cycle : coordinateur de vagues et spawning d'ennemis

Problème Le module `Wave` (coordinateur de vagues) doit déclencher l'apparition de nouveaux ennemis via `Soldier.add_random_soldier`. Mais `Soldier` doit notifier `Wave` de chaque nouvelle entité créée (`Wave.enemy_spawned`) et de sa mort (finaliseur `Wave.enemy_died`). Le cycle est :

$$\text{Soldier} \rightarrow \text{Wave} \rightarrow \text{Soldier}$$

Solution La solution repose sur une **référence de fonction** (*function pointer*), idiome classique pour rompre un couplage bidirectionnel sans recourir à des `module rec` ou des fichiers `.mli` supplémentaires.

```

1 (* Référence initialisée à un no-op, câblée depuis game.ml *)
2 let spawn_fn : (int -> unit) ref = ref (fun _ -> ())
3
4 let update () =
5   if !pending > 0 && !spawn_timer <= 0.0 then begin
6     let dir = Random.int 4 in
7     !spawn_fn dir;          (* appel indirect *)
8     decr pending;
9     spawn_timer := spawn_interval
10  end ...

```

Listing 8 – wave.ml : spawn_fn découple Wave de Soldier

```

1 let init dt =
2   Wave.spawn_fn := (fun dir -> ignore (Soldier.add_random_soldier dir));
3   ...

```

Listing 9 – game.ml : câblage au moment de l’initialisation

Wave ne dépend plus de Soldier à la compilation. Le câblage est effectué dans `game.ml`, point d’entrée qui a accès aux deux modules sans créer de cycle.

3.4 Bilan

Ces deux problèmes illustrent une tension fréquente en ECS : les systèmes de haut niveau (attaque, vagues) ont naturellement tendance à former des cycles avec les entités qu’ils produisent ou observent. Les solutions employées — isolation dans un fichier séparé, et référence de fonction injectable — sont deux techniques complémentaires qui préservent la séparation des responsabilités sans modifier l’interface publique des modules concernés.

4 Tests effectués

4.1 Méthode de développement et de test

Le développement s'est fait de façon itérative, en ajoutant une fonctionnalité à la fois et en testant visuellement dans le navigateur à chaque étape. Cette approche est adaptée aux projets de jeu où les comportements sont intrinsèquement visuels et difficiles à tester unitairement de manière isolée.

4.2 Tests manuels par composant

Placement de tours Vérification manuelle que le placement est refusé quand :

- Le budget est insuffisant (pas de décrétement non autorisé).
- L'emplacement est occupé par une autre entité (`Building.can_place`).

Système de collision et résolution Le système de collision est $O(n^2)$ sur les paires d'entités enregistrées. Il a été testé en observant que les projectiles disparaissaient bien au contact des ennemis, et que les ennemis stoppaient leur déplacement au contact d'une tour.

Logique d'attaque des tours Chaque tour a été testée individuellement :

- **Archer** : vérification de la portée (l'archer ne tire pas si aucun ennemi n'est dans le rayon de 150 px) et du cooldown (60 images entre deux tirs).
- **Bombardier** : vérification que l'explosion AoE touche bien tous les ennemis dans le rayon de 50 px. La zone rouge de visualisation a permis de confirmer la correspondance entre l'affichage et le calcul.
- **Geleur** : vérification visuelle du ralentissement des ennemis dans la zone bleue. Le facteur 0.3 a été ajusté empiriquement pour être visible sans rendre le jeu trop facile.

Logging via Gfx.debug Pendant tout le développement, des appels à `Gfx.debug` ont été utilisés pour tracer les événements importants dans la console du navigateur : dégâts infligés, mort d'une entité, démarrage d'une vague, etc.

```
1 Gfx.debug "Archer dealt %.0f damage! Enemy HP: %.1f\n%!" dmg hp;  
2 Gfx.debug "Bomber dealt %.0f damage! Enemy HP: %.1f\n%!" dmg hp;  
3 Gfx.debug "GAME OVER!!\n%!";
```

Listing 10 – Exemples de logs d'événements en jeu

4.3 Tests du système de vagues

Le coordinateur de vagues (`wave.ml`) maintient deux compteurs : `pending` (ennemis restant à spawner) et `alive` (ennemis vivants). Le retour automatique en phase de construction a été vérifié dans plusieurs cas :

- Vague terminée par élimination de tous les ennemis.
- Vague passée avec le noyau survivant mais des tours détruites.
- Difficulté croissante vérifiée ($5 + 3n$ ennemis pour la vague n).

4.4 Test du sous-typage structurel

La vérification de type au moment de la compilation a servi de test d'intégration permanent : toute tentative d'enregistrer une entité incomplète dans un système produisait une erreur de type OCaml décrivant précisément le composant manquant. Par exemple, retirer le champ `health` de la classe `projectile` générerait immédiatement :

```
Error: This expression has type projectile
      but an expression of type collidable was expected
      The method health has type unit but is expected to
      have type float Component.t
```

Ce mécanisme a évité de nombreux bugs potentiels à l'exécution sans écrire de tests unitaires supplémentaires.

4.5 Problèmes rencontrés non résolus

Un crash du jeu (gel complet de l'onglet navigateur) est apparu de façon intermittente, en général pendant ou après la première vague. Plusieurs hypothèses ont été examinées : suppression d'entités pendant une itération sur une table de hachage (invalidation d'itérateur), accumulation de projectiles n'ayant pas trouvé de cible et jamais supprimés, redimensionnement de la table pendant un parcours lazy `Seq`. Faute de temps, ce bug n'a pas été corrigé.